CS652/752
Advanced Algorithms and Applications

Dr. Xi Li

Spring 2025

Department of Computer Science
University of Alabama at Birmingham

Table of contents

Graph Algorithms

1. Graphs

2. Breadth-First Search (BFS)

3. Depth-First Search (DFS)

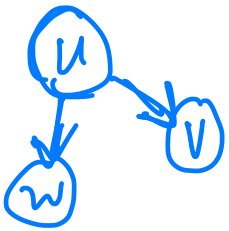
4. Minimum Spanning Trees (MST)

5. Prim's Greedy MST Algorithm

Graphs

Graphs

$V = \{u, v, w\}$

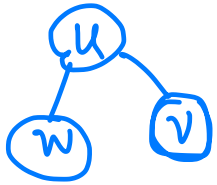


$E = \{(u,v), (u,w)\}$

Definition. A ***directed graph (digraph)*** $G = (V, E)$ is an ordered pair consisting of

- a set V of ***vertices*** (synonym: ***nodes***),
- a set $E \subseteq V \times V$ of ***edges***
- An edge $e=(u,v)$ goes "from u to v" (may or may not allow $u=v$)

- In an ***undirected graph*** $G = (V, E)$, the edge set E consists of *unordered* pairs of vertices
  - Sometimes write $e=\{u,v\}$

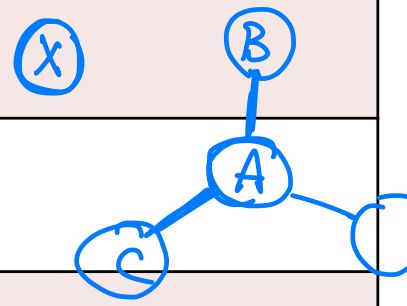


- Sometimes, store additional info: edge lengths, weights
- How many edges can a graph have?
  - In either case, $|E| = O(V^2)$.

A. Smith; based on slides by E. Demaine, C. Leiserson, S. Raskhodnikova, K. Wayne

Graphs are everywhere

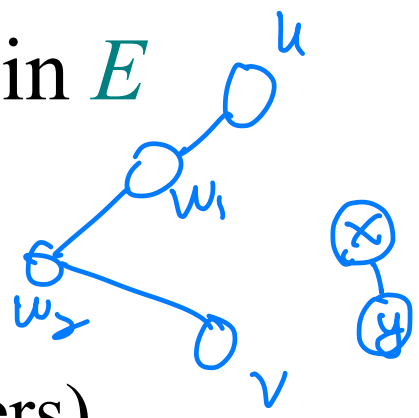
Example	Nodes	Edges
Transportation network: airline routes	airports	nonstop flights
Communication networks	computers, hubs, routers	physical wires
Information network: web	pages	hyperlinks
Information network: scientific papers	articles	references
Social networks	people	"u is v's friend", "u sends email to v", "u's MySpace page links to v"
Computer programs	functions (or modules) statement blocks	"u calls v" "v can follow u"



A. Smith; based on slides by E. Demaine, C. Leiserson, S. Raskhodnikova, K. Wayne

Paths and Connectivity

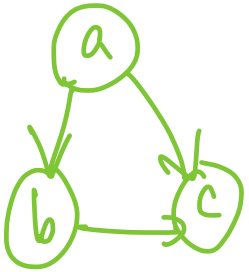
- **Path** = sequence of consecutive edges in E
  - $(u,w_1), (w_1,w_2), (w_2,w_3), \ldots, (w_{k-1}, v)$
  - Write $u \leftrightsquigarrow v$ or $u \rightsquigarrow v$
  - (Note: in a directed graph, direction matters)
  - Path is simple if no vertex is repeated



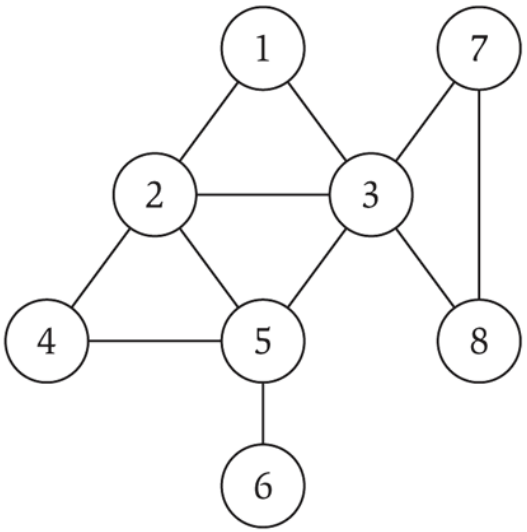
- Undirected graph G is **connected** if for every two vertices u,v, there is a path from u to v in G
- Directed graphs:
  - **Strongly connected** if for every pair, $u \rightsquigarrow v$ and $v \rightsquigarrow u$

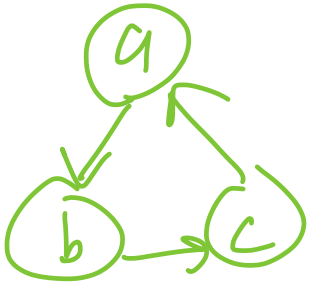
Cycles

- Def. A cycle is a path $v_1, v_2, \ldots, v_{k-1}, \cancel{v_k}$ v_1 in which $v_1 = v_k$, $k > 2$, and the first k-1 nodes are all distinct.



cycle C = 1-2-4-5-3-1





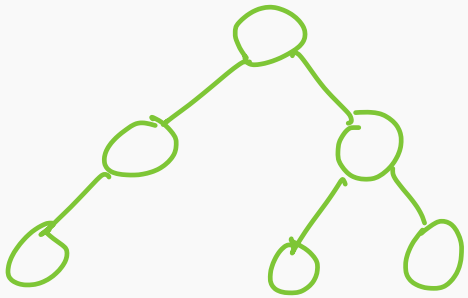
Graphs vs. Trees

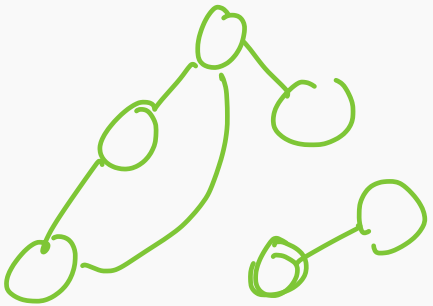
Trees: $\in$ graphs.

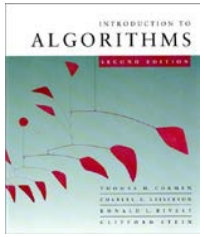
root

Feature	Tree	Graph
Structure	Hierarchical	General
Cycles	No	May have
Connectivity	Always connected	May be disconnected
Edges	$n - 1$ for n nodes	Varies
Path	One unique path	Possibly multiple paths

Table 1: Comparison between Trees and Graphs





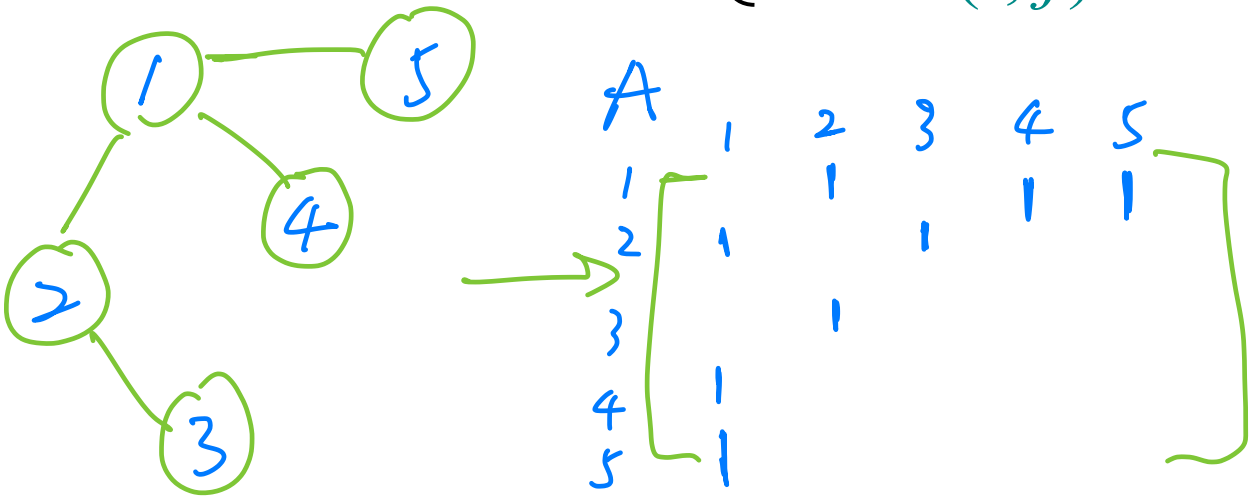


Adjacency-matrix representation

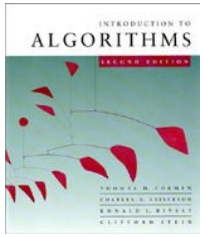
The ***adjacency matrix*** of a graph $G = (V, E)$, where $V = \{1, 2, \ldots, n\}$, is the matrix $A[1 \ldots n, 1 \ldots n]$ given by

$n \times n$

$$A[i, j] = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{if } (i, j) \notin E. \end{cases}$$



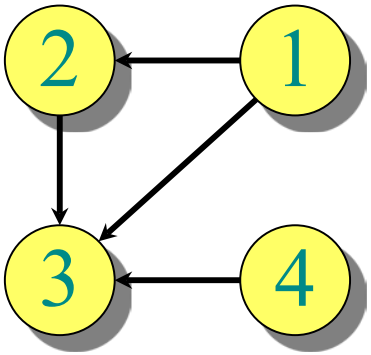
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Adjacency-matrix representation

The ***adjacency matrix*** of a graph $G = (V, E)$, where $V = \{1, 2, \ldots, n\}$, is the matrix $A[1 \,.\,.\, n, 1 \,.\,.\, n]$ given by

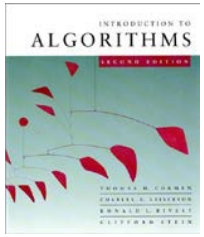
$$A[i, j] = \begin{cases} 1 & \text{if } (i, j) \in \mathrm{E}, \\ 0 & \text{if } (i, j) \notin \mathrm{E}. \end{cases}$$



A	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	1	0

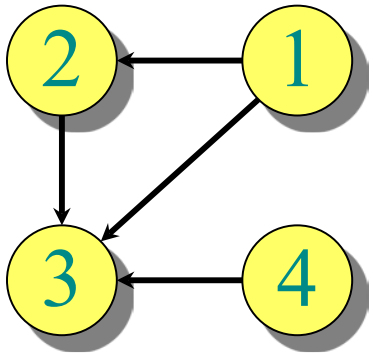
$\Theta(V^2)$ storage $\Rightarrow$ ***dense*** representation.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Adjacency-list representation

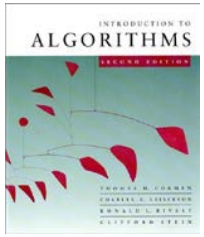
An ***adjacency list*** of a vertex $v \in V$ is the list $Adj[v]$ of vertices adjacent to v.



$Adj[1] = \{2, 3\}$
$Adj[2] = \{3\}$
$Adj[3] = \{\}$
$Adj[4] = \{3\}$

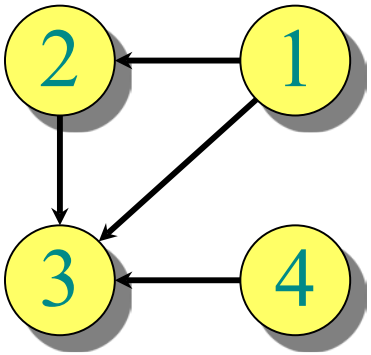
hash table.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



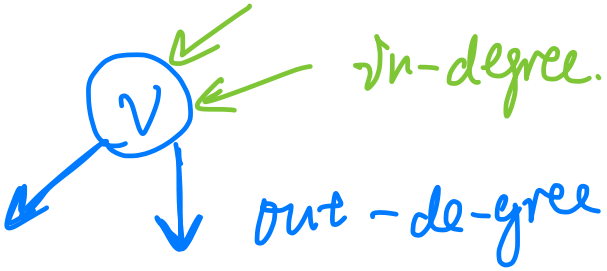
Adjacency-list representation

An ***adjacency list*** of a vertex $v \in V$ is the list $Adj[v]$ of vertices adjacent to v.

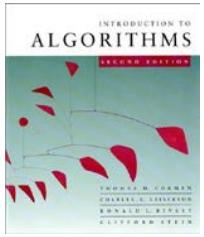


$Adj[1] = \{2, 3\}$
$Adj[2] = \{3\}$
$Adj[3] = \{\}$
$Adj[4] = \{3\}$

For undirected graphs, $|Adj[v]| = degree(v)$.
For digraphs, $|Adj[v]| = out\text{-}degree(v)$.

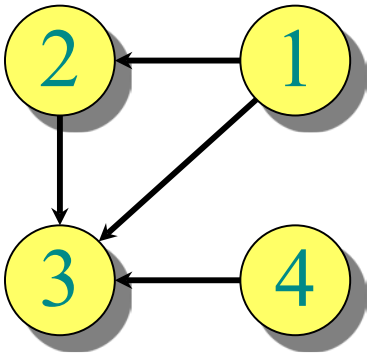


 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Adjacency-list representation

An ***adjacency list*** of a vertex $v \in V$ is the list $Adj[v]$ of vertices adjacent to v.



$Adj[1] = \{2, 3\}$
$Adj[2] = \{3\}$
$Adj[3] = \{\}$
$Adj[4] = \{3\}$

For undirected graphs, $|Adj[v]| = degree(v)$.
For digraphs, $|Adj[v]| = out\text{-}degree(v)$.

Handshaking Lemma: $\sum_{v \in V} degree(v) = 2|E|$ for undirected graphs $\Rightarrow$ adjacency lists use $\Theta(V + E)$ storage — a ***sparse*** representation. $\Theta(n+m)$

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Exploring a graph

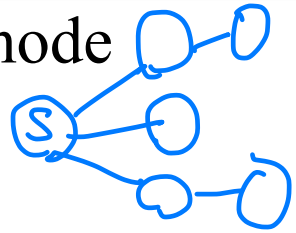
Classic problem: Given vertices $s,t \in V$, is there a path from s to t?

Idea: explore all vertices reachable from s

Two basic techniques:

- Breadth-first search (BFS)
  - Explore children in order of **distance** to start node
- Depth-first search (DFS)
  - Recursively explore vertex's children before exploring siblings

How to convert these descriptions to precise algorithms?

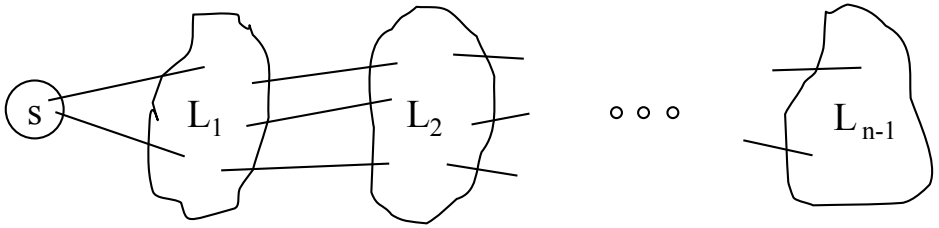


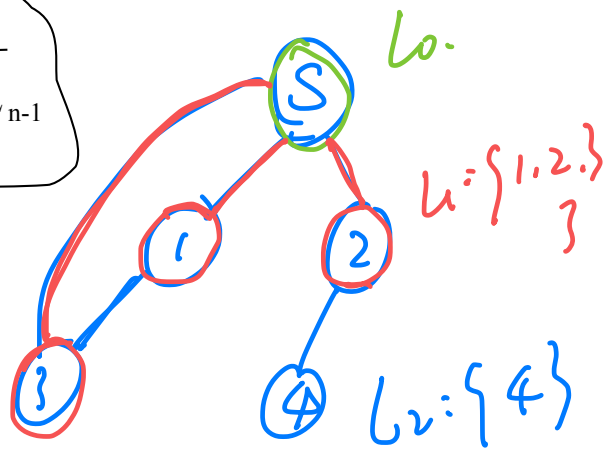
A. Smith; based on slides by E. Demaine, C. Leiserson, S. Raskhodnikova, K. Wayne

Breadth-First Search (BFS)

Breadth First Search

- BFS intuition. Explore outward from s in all possible directions, adding nodes one "layer" at a time.

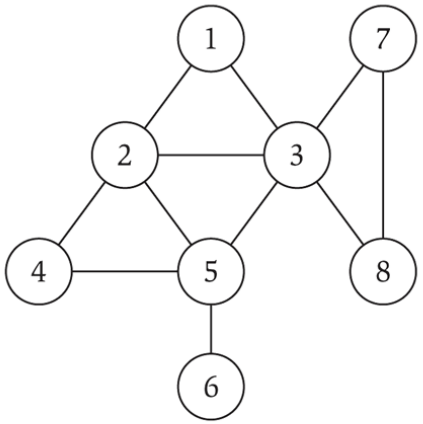


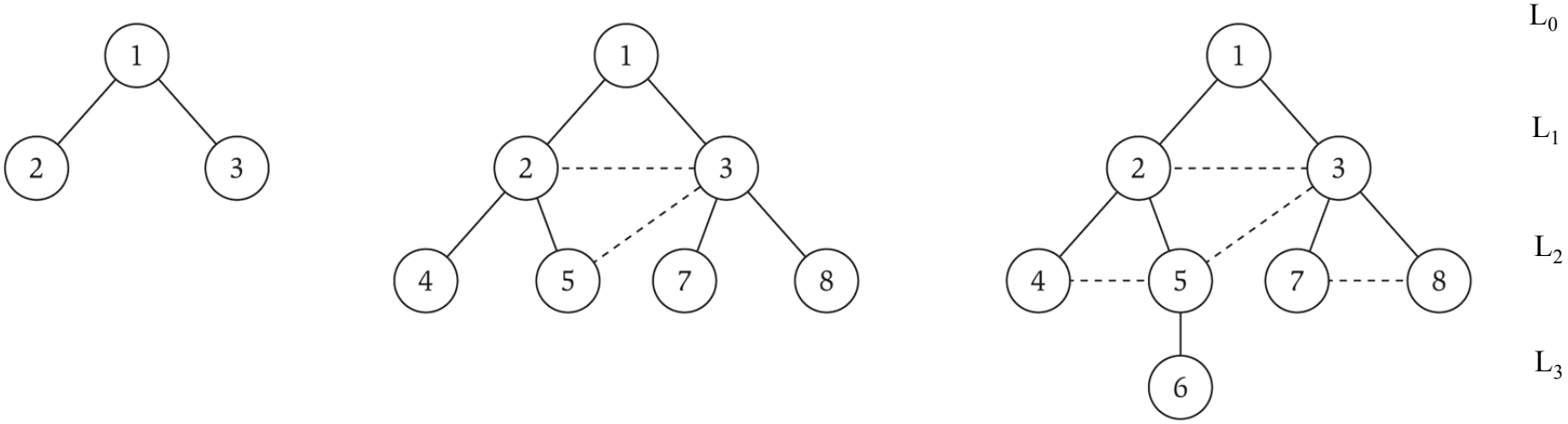


- BFS algorithm.
  - $L_0 = \{ s \}$.
  - L_1 = all neighbors of L_0.
  - L_2 = all nodes that do not belong to L_0 or L_1, and that have an edge to a node in L_1.
  - L_{i+1} = all nodes that do not belong to an earlier layer, and that have an edge to a node in L_i.

Breadth First Search

- Property. Let T be a BFS tree of G = (V, E), and let (x, y) be an edge of G. Then the level of x and y differ by at most 1.





(a) (b) (c)

BFS with adjacency list rep.

- We start with an intuitive, but somewhat inefficient version of BFS. The book presents a more streamlined solution:

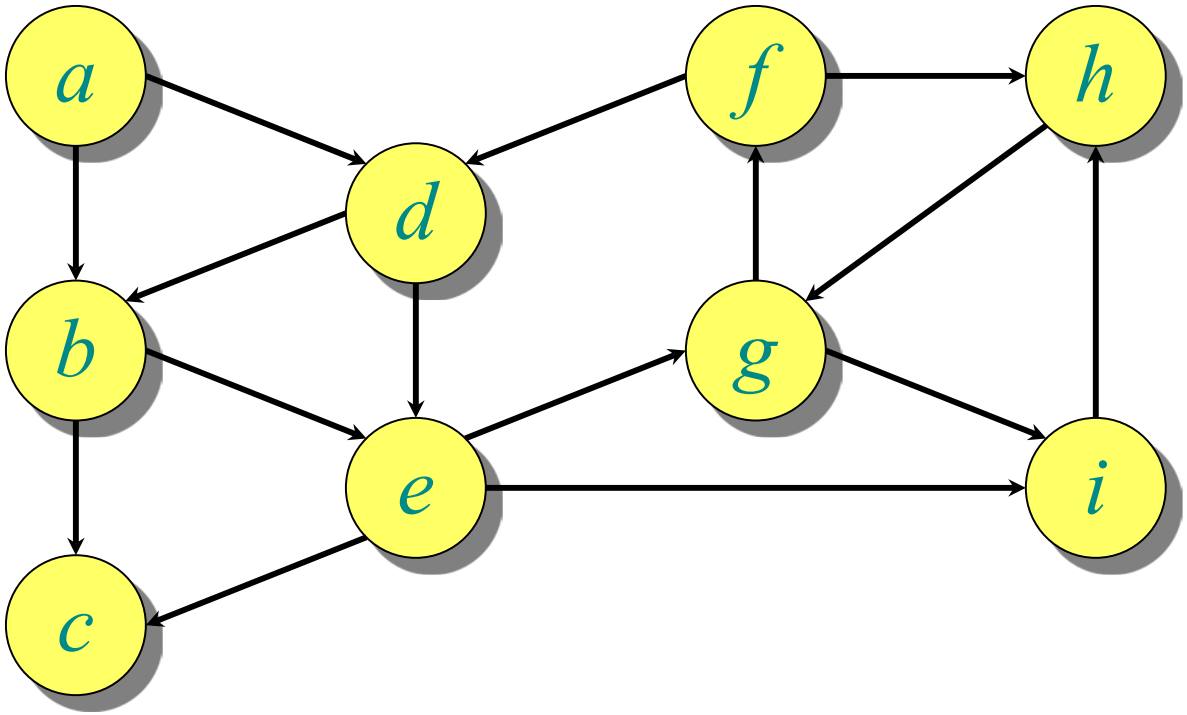
Data structures used

- Level[1 .. n]:
  - array of integers
  - (Or hash table where value = level)
- Queue Q
  - initialized to empty
- Tree T
  - initialized to empty
  - Could be just a set of edges
  - Or could simply store the parent pointer for each vertex

CLRS code

```
BFS(V, E, s)
for each u ∈ V − {s}
    u.d = ∞
s.d = 0
Q = ∅
ENQUEUE(Q, s)
while Q ≠ ∅
    u = DEQUEUE(Q)
    for each v ∈ G.Adj[u]
        if v.d == ∞
            v.d = u.d + 1
            ENQUEUE(Q, v)
```

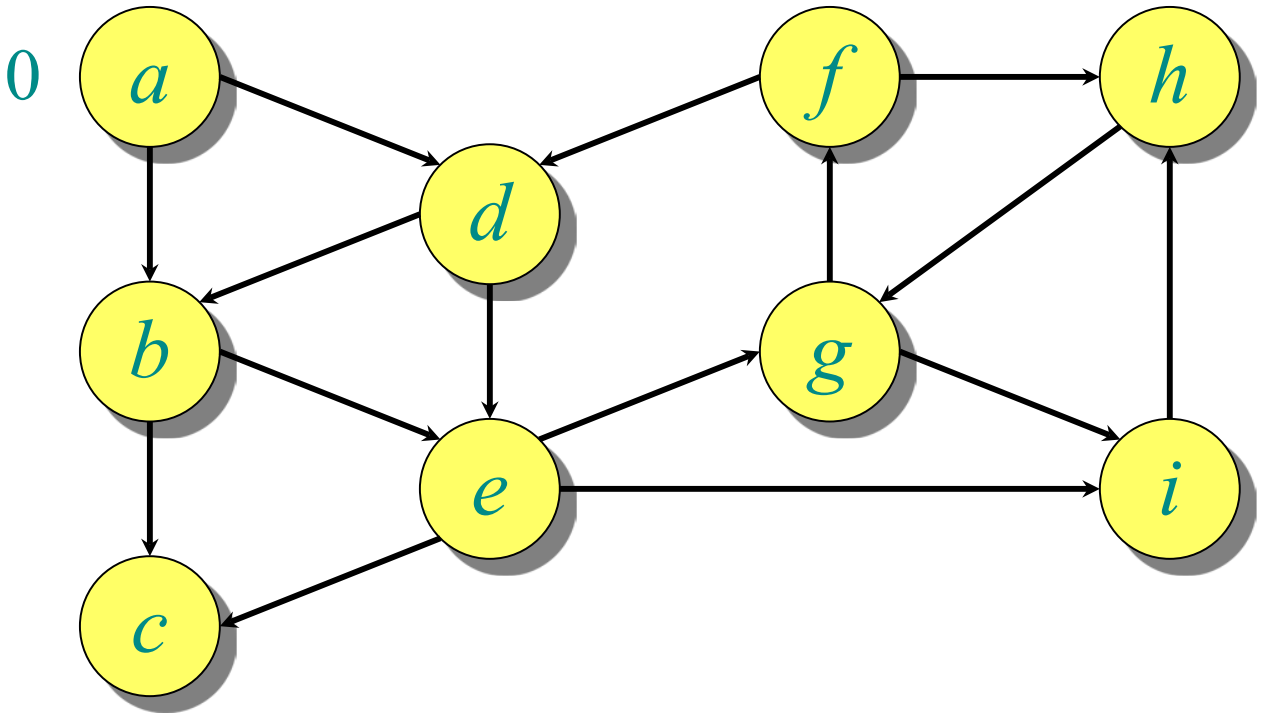
Example of breadth-first search



Q:

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

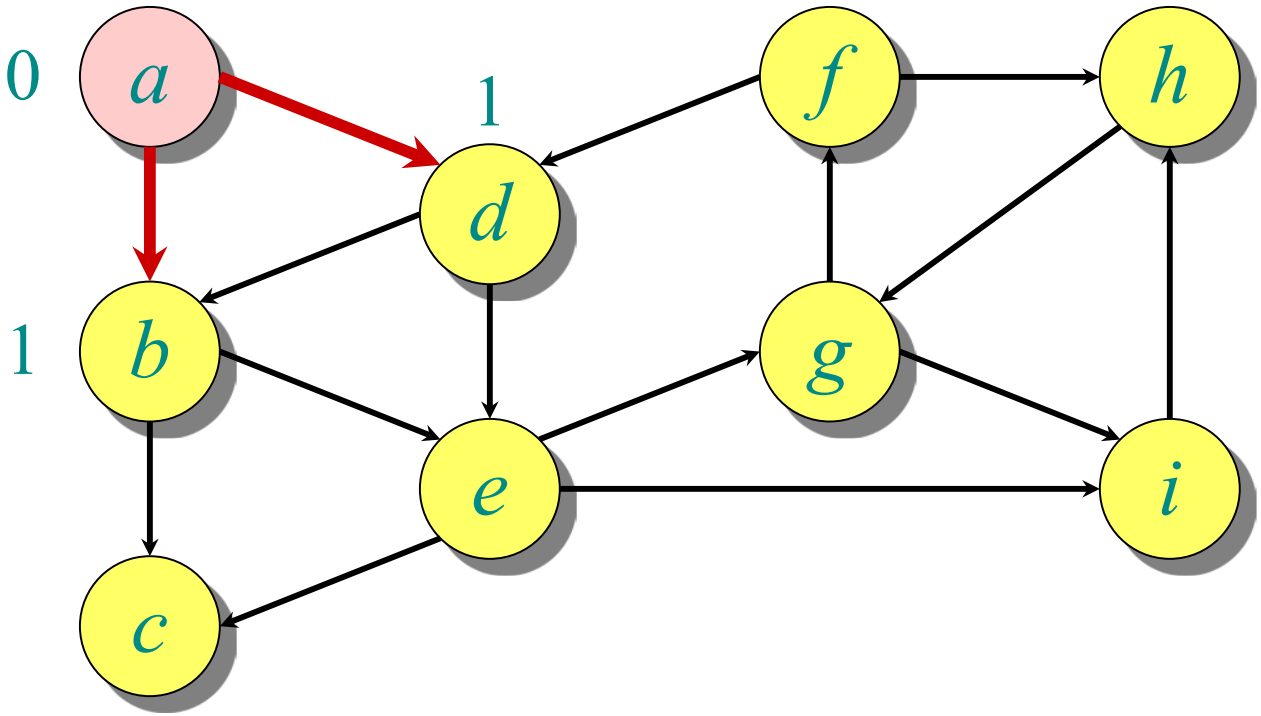


0

Q: a

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

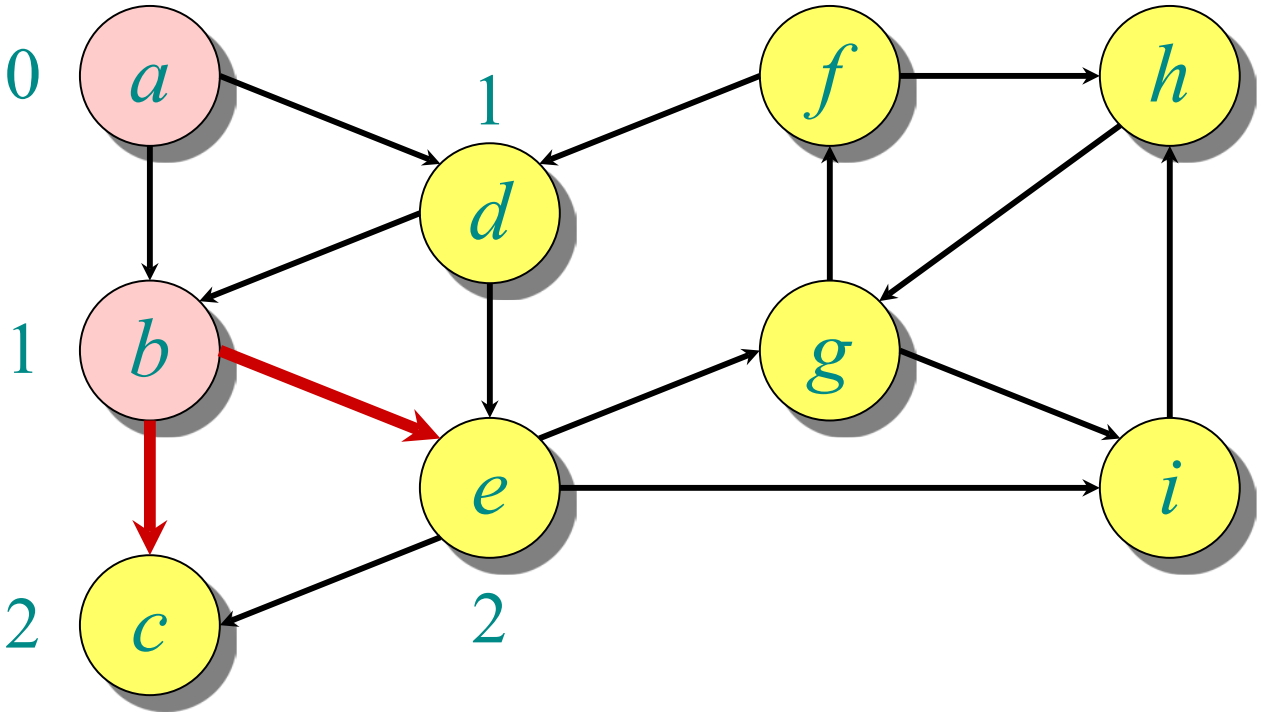


1 1

Q: *a* *b* *d*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

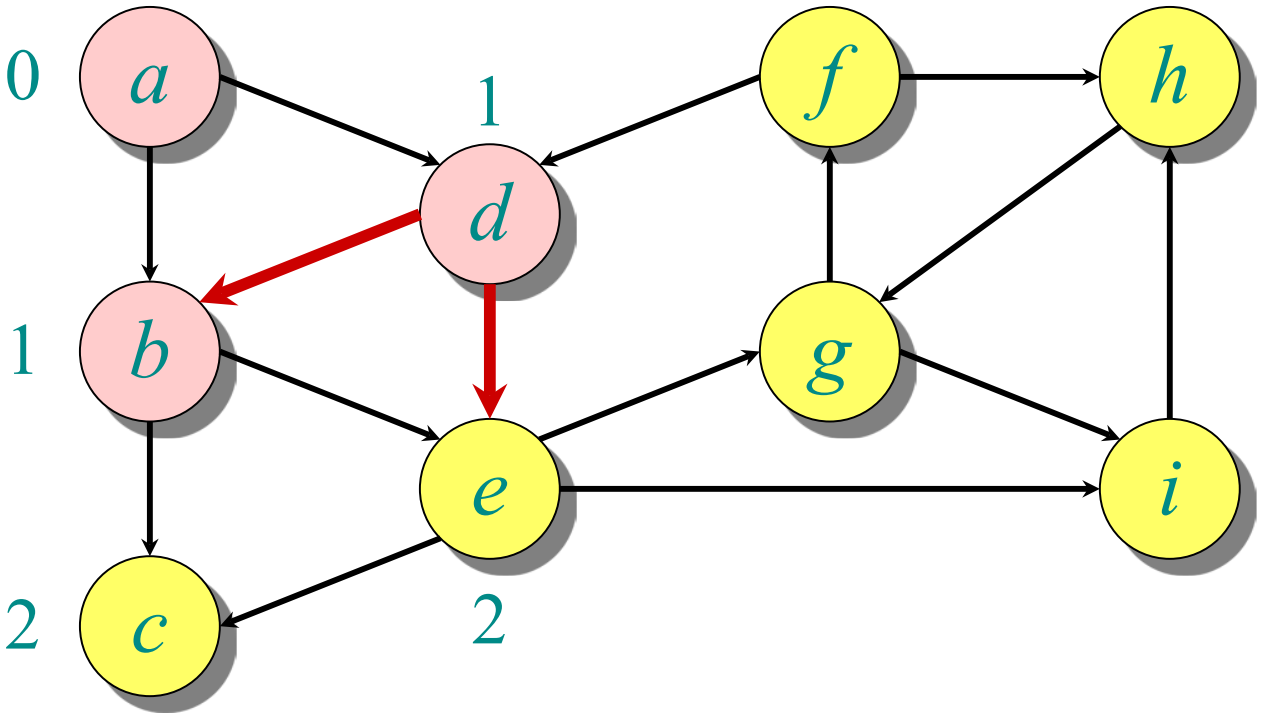


1 2 2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

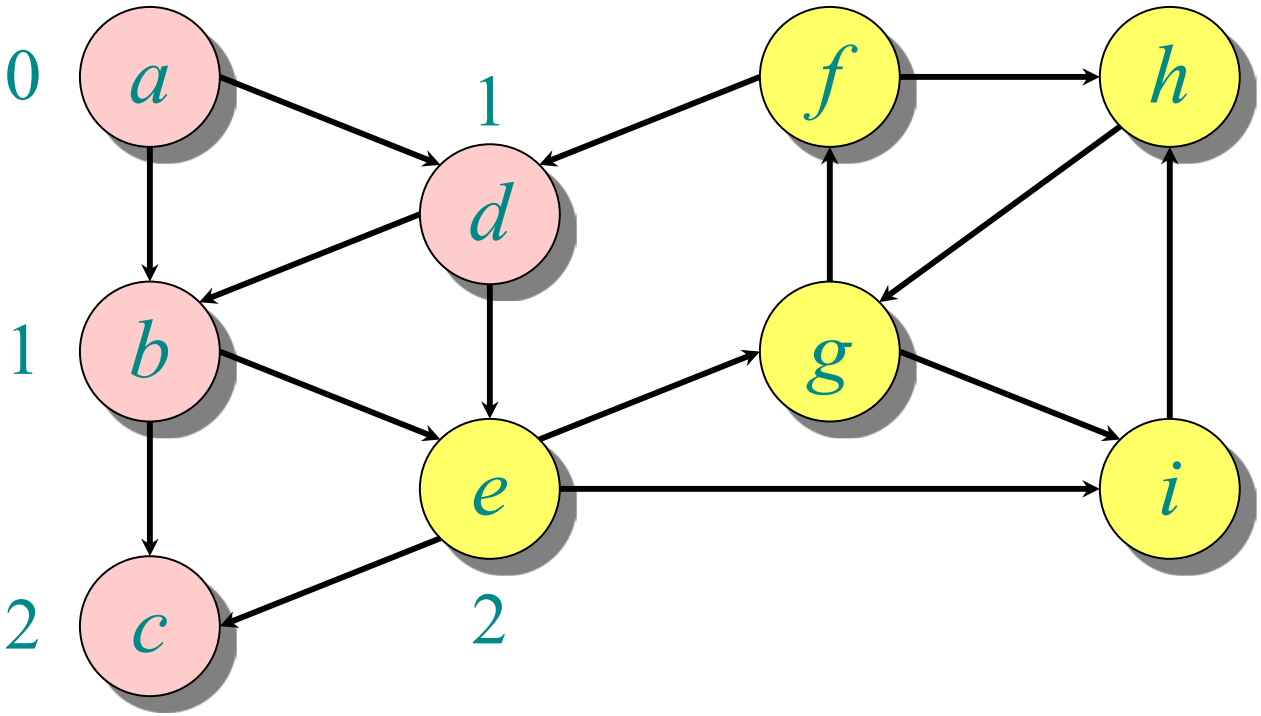


2 2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

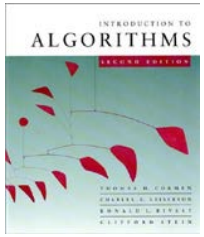
Example of breadth-first search



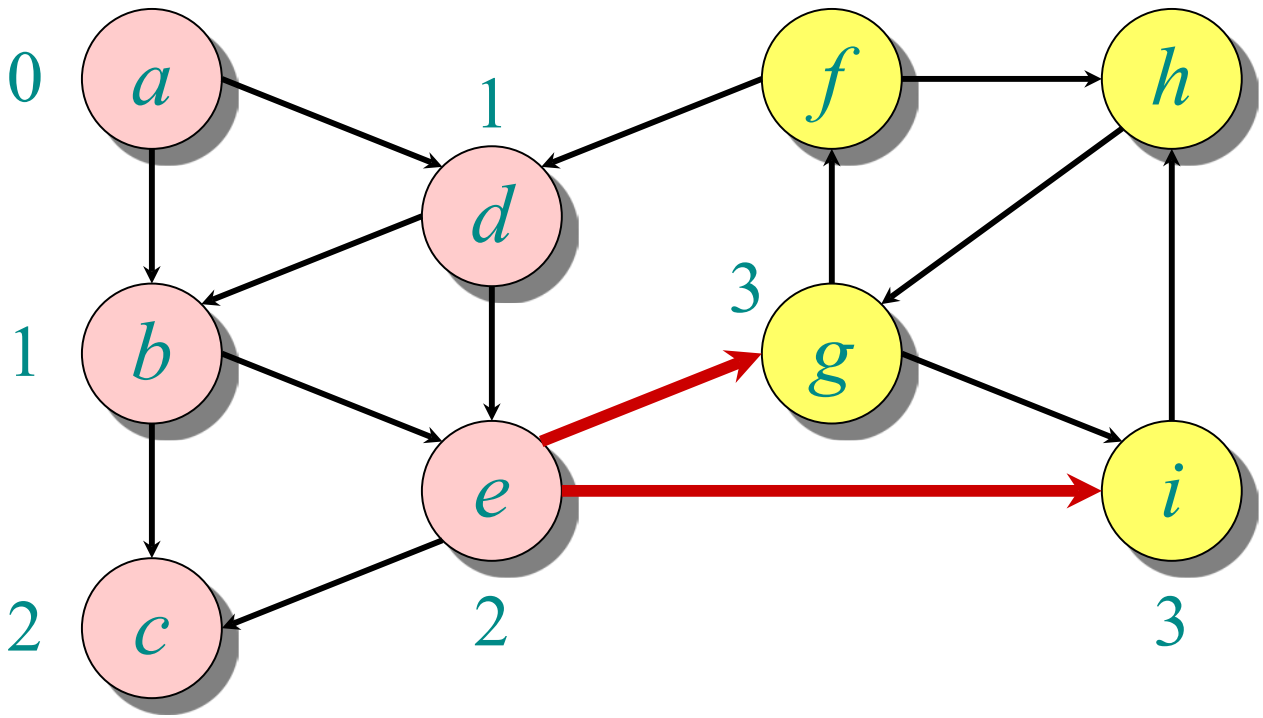
2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of breadth-first search

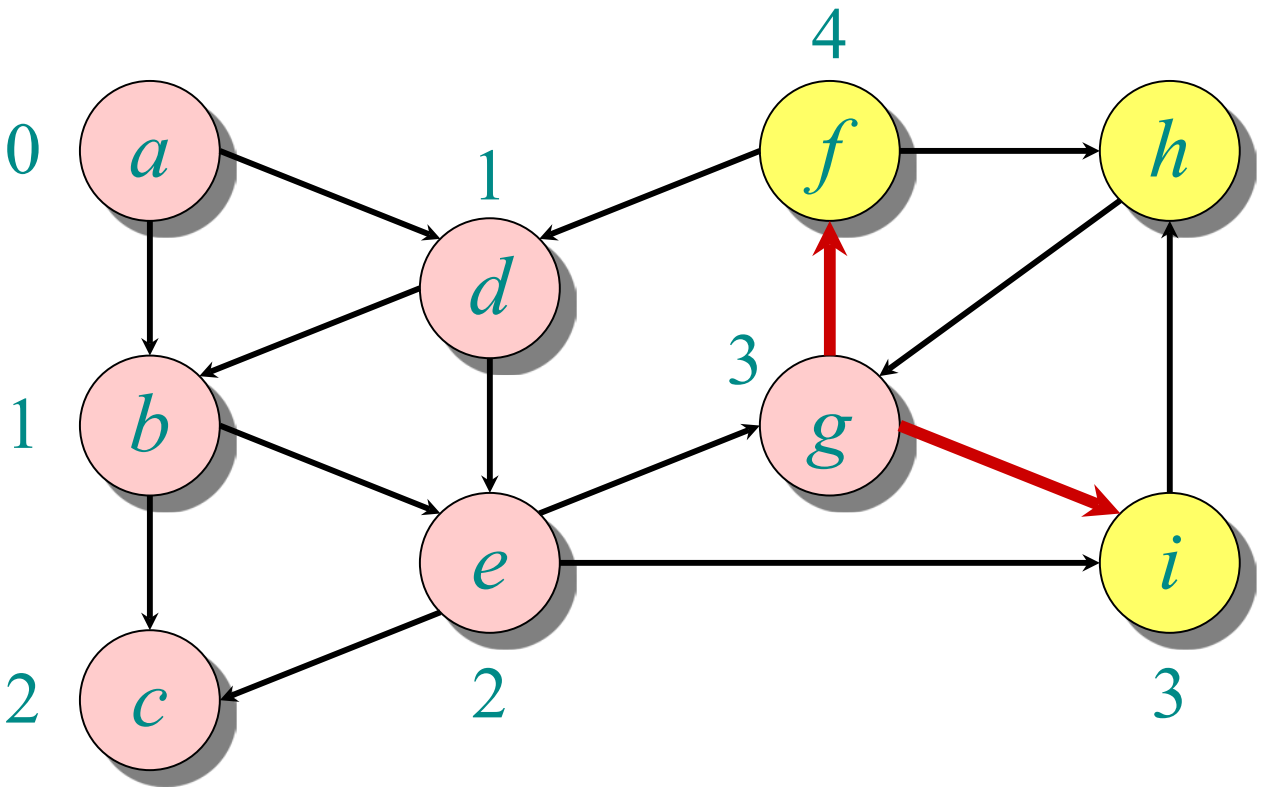


3 3

Q: *a* *b* *d* *c* *e* *g* *i*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

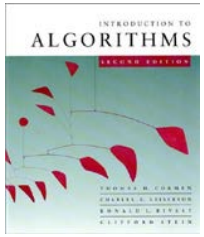
Example of breadth-first search



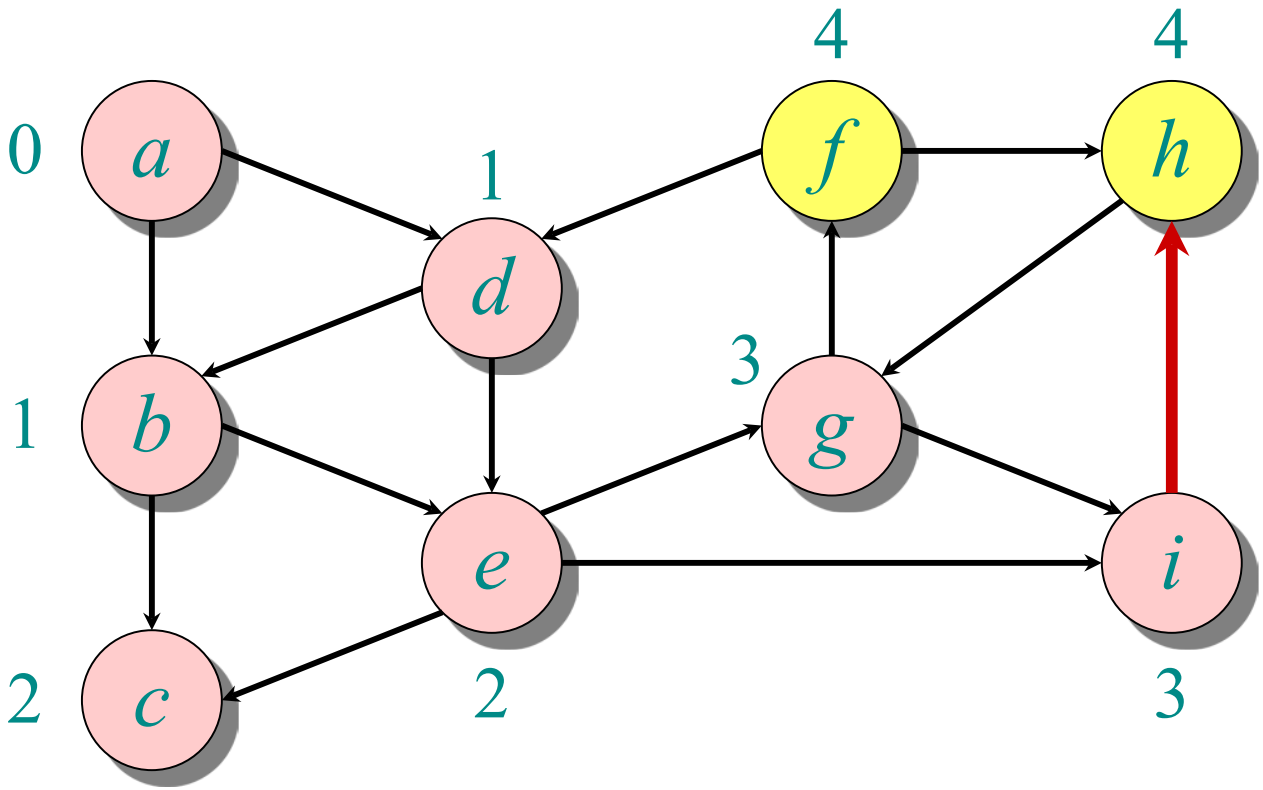
3 4

Q: *a* *b* *d* *c* *e* *g* *i* *f*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Example of breadth-first search

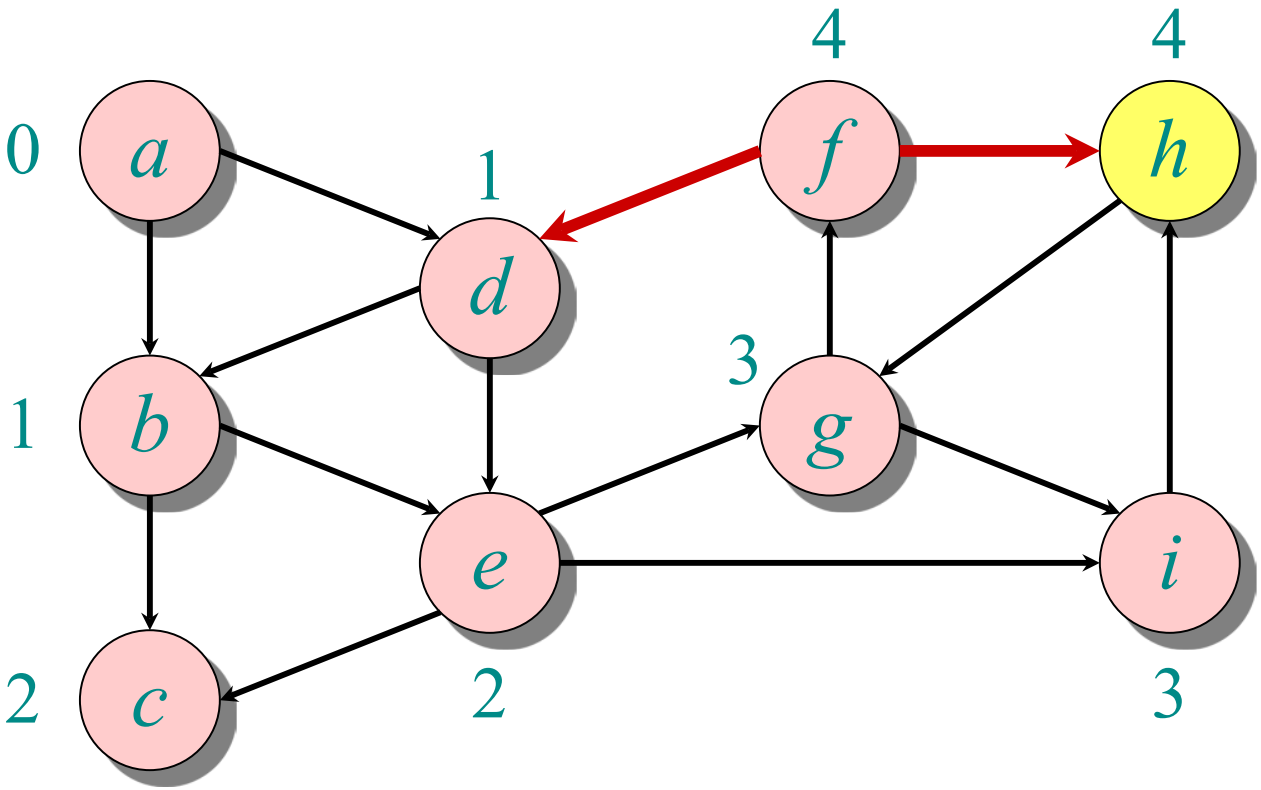


4 4

Q: *a* *b* *d* *c* *e* *g* *i* *f* *h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

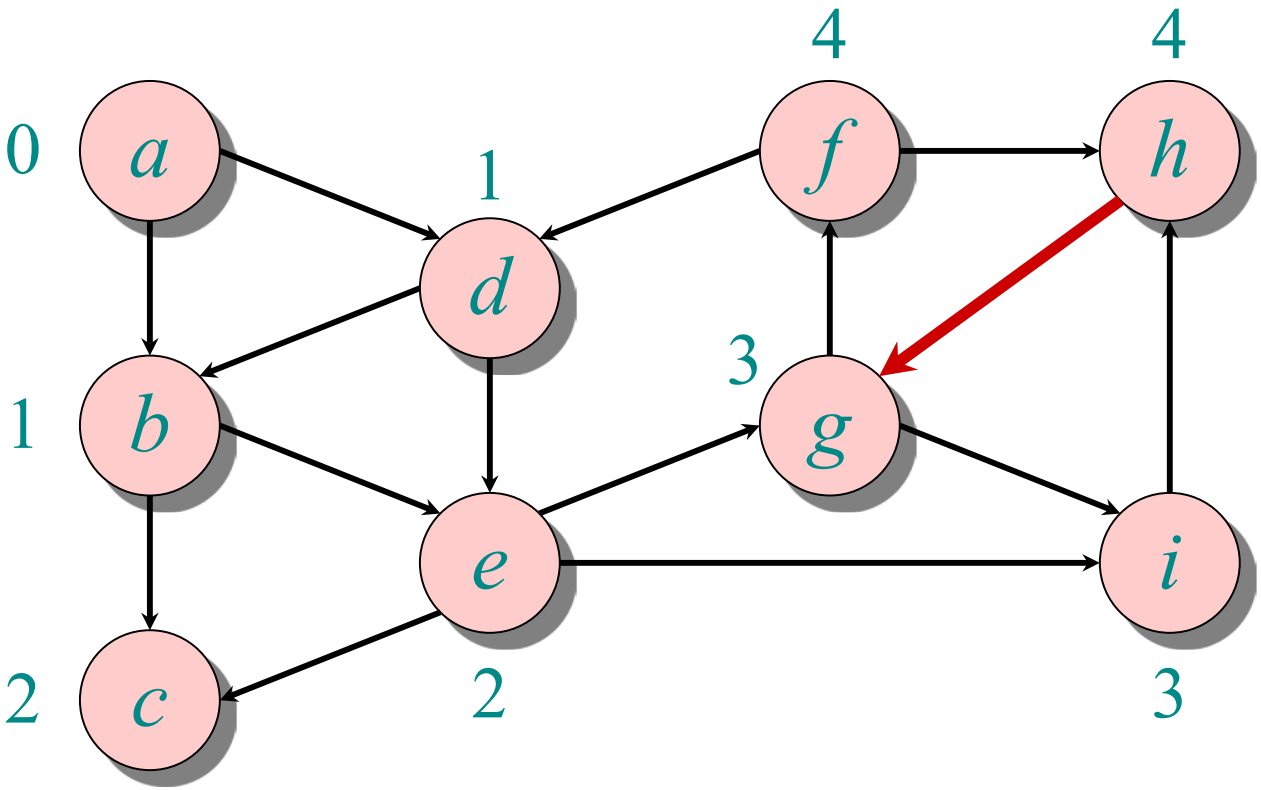


4

Q: *a* *b* *d* *c* *e* *g* *i* *f* *h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

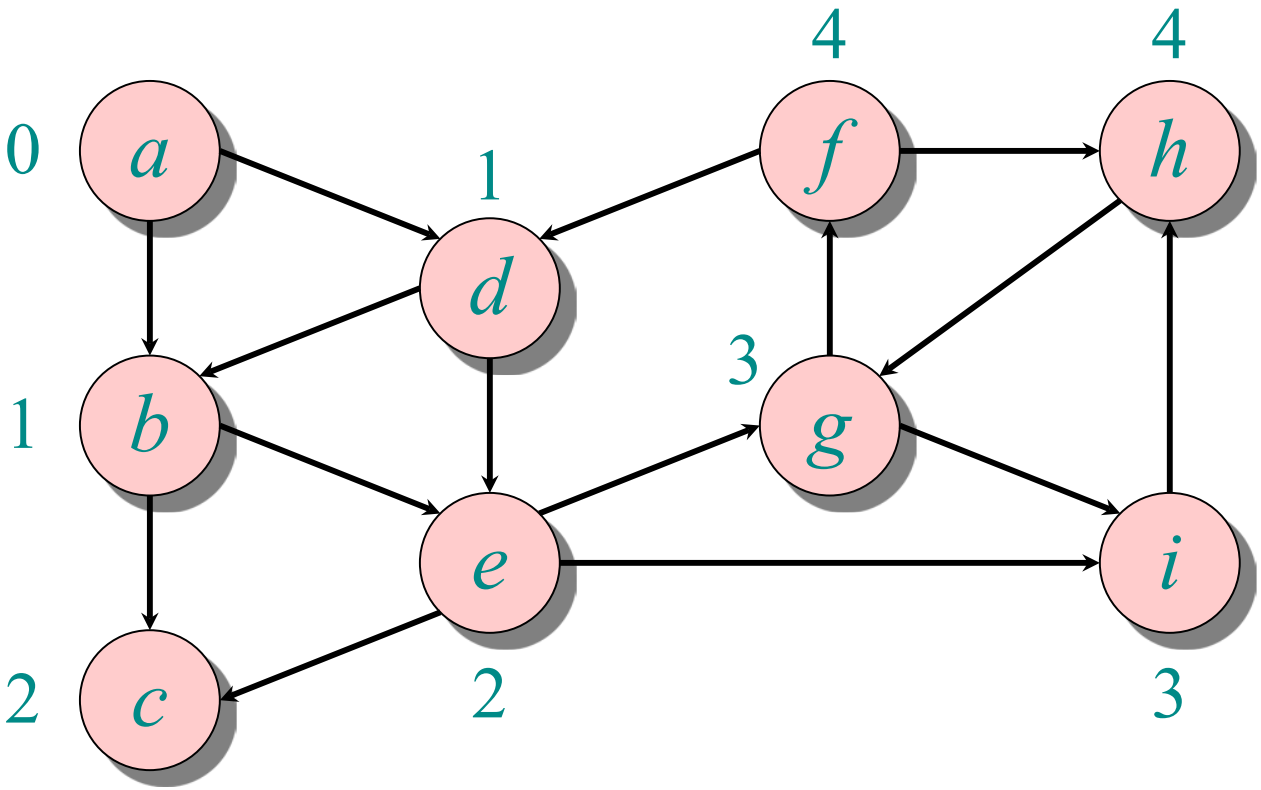
Example of breadth-first search



Q: *a b d c e g i f h*

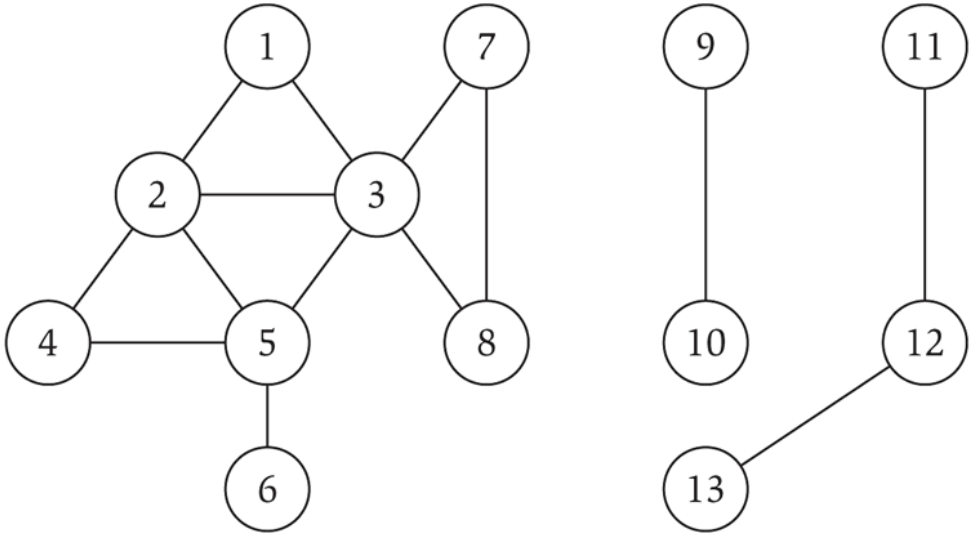
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search



Q: *a b d c e g i f h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Application: connected component

- Connected component
  - set of nodes all of which can reach each other
- Every undirected graph can be broken up into disjoint connected components
  - BFS finds all nodes in connected component of s